USER GUIDE

CompLB3D

A 3D pore-scale reactive transport model
with microbial metabolism

Version 1.0.0

Shahram Asgari • Christof Meile

Department of Marine Sciences
University of Georgia, Athens, GA, USA

shahram.asgari@uga.edu

CompLB3D extends the two-dimensional CompLaB, developed by Heewon Jung (Chungnam National University) and co-workers with the University of Georgia.

How to read this guide

This guide is written to be followed in order, not sampled. It is divided into four parts and they do different jobs.

Part I	**Getting started.** Install the code, build it, run one simulation, and understand what came out. Read this once, in order. Nothing later makes sense without it.
Part II	**Tutorial.** One chapter per capability, each built on a worked example that ships with the code and runs in seconds. Read the chapters you need.
Part III	**Going further.** Writing your own chemistry, fitting a surrogate, running on a cluster, post-processing.
Part IV	**Reference.** Every XML tag with its type, default and units. Every error message with its cause and fix. Look things up here.

If you have thirty minutes, read Chapter 2 and Chapter 3, and run the first example. That is enough to know whether this code does what you need.

If you are here to do one specific thing, the map is:

I want to . . .	Go to . . .
run my own pore geometry	Chapter 4, Section 4.3.2
write my own rate laws	Chapter 19
use a genome-scale metabolic model	Chapter 10
make my simulation faster than FBA	Chapter 11
model pore clogging or mineral loss	Chapter 13
run on a cluster	Chapter 21
understand an error message	Chapter 24
look up one tag	Chapter 23

Conventions. Commands you type are shown in a shaded box. `<tag>` means an element of `CompLaB.xml`. A boxed note in teal is something worth knowing; a boxed note in red is something that will cost you a day if you get it wrong.

Contents

Part I

Getting started

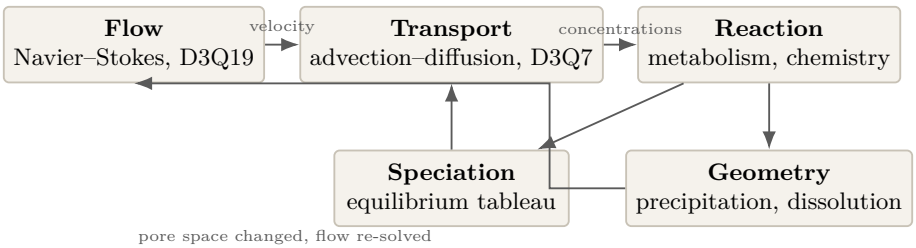
CHAPTER 1

What CompLB3D does

CompLB3D simulates what happens in the pore space of a sediment, a soil or a porous rock: water flows through it, dissolved chemicals move with the water, microbes eat them and grow, minerals form and dissolve, and the pore space itself changes shape as a result.

It does this at the *pore scale*. The grid is fine enough that individual grains and pore throats are resolved, so you are not fitting an effective diffusivity or a tortuosity factor: the geometry is in the model, and transport follows from it.

1.1 The five things it couples



Every box except the first two is optional and switched on from the input file. With all of them off, CompLB3D is a flow-and-transport solver and nothing more. That is deliberate: you should be able to turn one thing on at a time and see what it did.

1.2 What makes it different from the 2D CompLaB

CompLB3D began as a port of CompLaB to three dimensions and grew several capabilities the 2D code does not have.

	CompLaB (2D)	**CompLB3D**
Dimensions	D2Q9 / D2Q5	D3Q19 / D3Q7
Biomass solvers	CA, FD, LBM	CA, FD, LBM
Metabolism	kinetics, FBA, surrogate	the same, plus combined types
Aqueous speciation	—	equilibrium tableau
Abiotic kinetics	—	separate hook
Precipitation	—	volume-of-pixel pore clogging
Dissolution	—	pore reopening
Surrogate training	not distributed	MATLAB and Python paths
Worked examples	6, one problem	15, one per capability

1.3 What it is not

Being honest about this saves time.

- **It is not a continuum reactive transport code.** There is no Darcy law and no representative elementary volume. If your domain is metres across, this is the wrong tool.
- **The speciation is ideal.** No activity corrections, no temperature dependence, no gas phase, no redox couples, no mineral saturation indices. The equilibrium constants you supply are conditional constants at your own ionic strength.
- **There is no unsaturated flow.** One fluid phase only.
- **Rate laws are compiled in.** Changing your chemistry means editing a C++ header and rebuilding. This surprises people who expect an input file to be enough.

CHAPTER 2

Installing and building

2.1 What you need

Component	Required?	Notes
C++ compiler	yes	GCC or Clang, C++11 or later
CMake	yes	3.10 or later
Palabos	yes	**version 2.3.0**. Downloaded separately.
MPI	recommended	on by default; turn off with `-DENABLE_MPI=OFF`
GLPK	optional	only for flux balance analysis through GLPK
Python 3 + cobrapy	optional	only for flux balance analysis through COBRApy
ParaView	recommended	to look at the output

> **Palabos version matters.** CompLB3D is developed against Palabos **v2.3.0**. Later versions change data-processor interfaces, and the code will not compile against them without edits. Get 2.3.0.

2.2 Step 1: get Palabos

Palabos is a lattice-Boltzmann framework and it is a separate work under its own licence. It is not distributed with CompLB3D, partly because it is around 100 MB and partly because bundling someone else's AGPL library inside yours muddies the licence question for everybody downstream.

Download `palabos-v2.3.0` from `https://palabos.unige.ch/` and unpack it somewhere permanent.

```
tar xzf palabos-v2.3.0.tar.gz -C /your/path/
```

You do not have to build Palabos separately — CompLB3D compiles the parts it needs. But building it once on its own is worth the ten minutes, because if something is wrong with your compiler or MPI you will find out there, in a project with good documentation, rather than here.

```
cd /your/path/palabos-v2.3.0/build
cmake ..
make
```

Warnings during that build are normal and can be ignored as long as it finishes.

2.3 Step 2: get CompLB3D

```
git clone https://github.com/shahram444/CompLB3D-lattice-Boltzmann-FBA-surrogate-COBRApy-GLPK.
    git
```

```
cd CompLB3D-lattice-Boltzmann-FBA-surrogate-COBRApy-GLPK
```

2.4 Step 3: point CMake at Palabos

Open `CMakeLists.txt` and edit the five lines that name the Palabos tree. They are around lines 85 to 93.

```
include_directories("/your/path/palabos-v2.3.0/src")
include_directories("/your/path/palabos-v2.3.0/externalLibraries")
include_directories("/your/path/palabos-v2.3.0/externalLibraries/Eigen3")
file(GLOB_RECURSE PALABOS_SRC "/your/path/palabos-v2.3.0/src/*.cpp")
file(GLOB_RECURSE EXT_SRC "/your/path/palabos-v2.3.0/externalLibraries/tinyxml/*.cpp")
```

This is the single most common reason a first build fails. If CMake reports that it cannot find Palabos headers, this is why.

2.5 Step 4: build

```
mkdir build && cd build
cmake ..
make -j
```

That gives you the plain build: flow, transport, hand-written kinetics, speciation, precipitation and dissolution. It does *not* include flux balance analysis, because that needs an optional dependency.

2.5.1 Optional: flux balance analysis through GLPK

Install GLPK first — your package manager will have it (`apt install libglpk-dev`, `brew install glpk`, `module load glpk` on a cluster). Then:

```
cmake -DENABLE_GLPK=ON ..
make -j
```

2.5.2 Optional: flux balance analysis through COBRApy

This embeds a Python interpreter, so you need the Python *development* headers, not just the interpreter.

```
python3 -m pip install cobrapy
cmake -DENABLE_COBRAPY=ON ..
make -j
```

> **Both at once.** `cmake -DENABLE_GLPK=ON -DENABLE_COBRAPY=ON ..` builds an executable that can do either, chosen at run time from the input file. Worth doing once so you never think about it again.

2.5.3 The build options in full

Option	Default	Effect
`ENABLE_MPI`	ON	parallel build. Turn off for a single-core debug build.
`ENABLE_GLPK`	OFF	links GLPK; enables `reaction_type glpk`
`ENABLE_COBRAPY`	OFF	embeds Python; enables `reaction_type cobrapy`
`FBA_BULK_ONLY`	OFF	restricts the FBA processors to the bulk domain. An optimisation; leave it off until you have a reason.

2.6 Step 5: check the build

The fastest confidence check is the example suite, which ships with the code.

```
python3 examples/makeGeometry.py
python3 examples/makeExamples.py
python3 examples/validateExamples.py .
```

The last command checks the fifteen examples structurally — tag names, vector lengths, model file dimensions — and compiles their kinetics headers. It takes about a second and should end with:

```
15 case(s) checked, 0 failed, 0 note(s).
```

Then run them:

```
./examples/runAllExamples.sh .
```

2.7 When the build fails

Message	Cause and fix
`Cannot find palabos3D.h`	The paths in `CMakeLists.txt` are wrong. Step 3.
`ENABLE_GLPK is ON but GLPK was not found`	Install `libglpk-dev`, or drop the flag.
`ENABLE_COBRAPY is ON but the Python development headers were not found`	You have Python but not `python3-dev`. Install it.
undefined reference to `MPI_*`	Building with MPI but linking without it. Load the same MPI module you configured with, and delete `build/` before reconfiguring.
errors inside Palabos headers	Almost always the wrong Palabos version. Check it is 2.3.0.
`defineKinetics.hh: C[94] out of range`	You are compiling a kinetics header written for different chemistry. Chapter 19.

Delete `build/` when you change configuration. CMake caches its options. Switching `ENABLE_GLPK` on and off without clearing the cache produces builds that link the wrong things and fail in confusing ways.

CHAPTER 3

Your first simulation

We will run example 01. It is the smallest thing that exercises both solvers: water driven through a channel, and a dissolved tracer that rides along and reacts with nothing.

3.1 Run it

```
# from the top of the CompLB3D tree
cp examples/01_flow_only/defineKinetics.hh        .
cp examples/01_flow_only/defineAbioticKinetics.hh .
cd build && make -j && cd ..

mkdir -p run01 && cd run01
cp ../examples/01_flow_only/CompLaB.xml .
cp -r ../examples/01_flow_only/input .
mkdir -p output
../build/complab
```

Why the two `.hh` files get copied. CompLaB compiles its rate laws in. `defineKinetics.hh` and `defineAbioticKinetics.hh` are `#included`, not read at run time, so a case with different chemistry needs a different binary. This catches everyone once. See Chapter 19.

`CompLaB.xml` is read by that exact name from the current working directory. Not from the executable's directory, not from a path you pass on the command line. Run the code from the folder that holds the input file.

3.2 What you should see

The run prints a long banner at start-up. It is worth reading, because most input mistakes are visible in it. The parts that matter:

```
[OK] XML configuration loaded and validated

  domain          : 24 x 24 x 6   (3456 voxels)
  substrates      : 1
  microbes        : 0             (abiotic mode)
  ...
  [IMMOBILE] immobile (solid/precipitate) substrates: 0 of 1
```

then the flow solver converging, then the transport steps. At the end you have a folder of `.vti` files in `output/`.

3.3 Reading the run

Three things are worth checking on every run you ever do.

3.3.1 Did the flow converge?

The Navier–Stokes solver iterates until the velocity field stops changing by more than `<ns_converge_iT1>`, or until `<ns_max_iT1>` steps. If it hits the iteration cap instead of the tolerance, your flow field is not converged and everything downstream inherits that.

3.3.2 Are there `[NEG!]` warnings?

A negative concentration is not physical. It means a reaction removed more of something than was there, which usually means the time step is too long for the rate you specified. One or two at start-up while fields settle is tolerable; a stream of them is a real problem. Chapter 24.

3.3.3 Does the answer depend on z?

Every shipped example is $24 \times 24 \times 6$ with z periodic and the geometry extruded through the depth. That makes them quasi-2D on purpose: every z-plane sees identical conditions, so the answer *must not* depend on z. If it does, something is wrong with the setup, not with the physics. It is a free check and it costs nothing to look.

3.4 Looking at the output

CompLB3D writes VTK ImageData (`.vti`) files, one per field per save interval. Open them in ParaView:

1. **File** → **Open**, select the numbered group (ParaView collapses `subsLattice0_0000500.vti` and friends into one entry), and hit Apply.
2. Set the **Coloring** dropdown to the field you want.
3. Use **Slice** to cut through the domain; the raw view of a 3D block shows you only its outer surface.

> **The `.vti` files use unit spacing.** ParaView therefore shows lattice coordinates, not micrometres. Multiply by `<dx>` if you need physical distance. This is a known wart, not a setting.

3.5 What to try next

Change one thing and rerun. Good first experiments:

Change	**What should happen**
`<Peclet>` from 1 to 0	flow switches off; the tracer profile becomes a straight line. This is example 02.
`<Peclet>` from 1 to 10	advection dominates; the tracer front sharpens and moves faster.
`<delta_P>` up	higher velocity, and the flow solver takes longer to converge.
`<nz>` from 6 to 12	the answer must not change. If it does, tell us.

CHAPTER 4

The input file

Everything about a run is in `CompLaB.xml`. This chapter walks through its structure. Chapter 23 is the exhaustive tag list; this one is the map.

4.1 The six blocks

`<path>`	where the input and output folders are
`<simulation_mode>`	which physics is switched on
`<LB_numerics>`	domain, geometry, time stepping
`<chemistry>`	the solutes: one block each
`<microbiology>`	the organisms: one block each
`<equilibrium>`, `<precipitation>`, `<dissolution>`, `<IO>`	optional features and output

4.2 `<simulation_mode>`: the master switches

This block decides which physics runs at all. Everything in it is a `true` or `false`.

```
<simulation_mode>
    <biotic_mode>true</biotic_mode>
    <enable_kinetics>true</enable_kinetics>
    <enable_abiotic_kinetics>false</enable_abiotic_kinetics>
    <enable_fba_glpk>false</enable_fba_glpk>
    <enable_fba_cobrapy>false</enable_fba_cobrapy>
    <enable_surrogate>false</enable_surrogate>
</simulation_mode>
```

`<biotic_mode>` is the big one: `false` skips the whole `<microbiology>` block, and CompLB3D becomes a chemistry-and-transport code.

The three metabolic switches are *global*. Each microbe then chooses its own `<reaction_type>`, and the two have to agree — a microbe asking for `glpk` while `<enable_fba_glpk>` is `false` stops the run with a message. That double gate is deliberate: it means you cannot accidentally get different biology from the one you asked for.

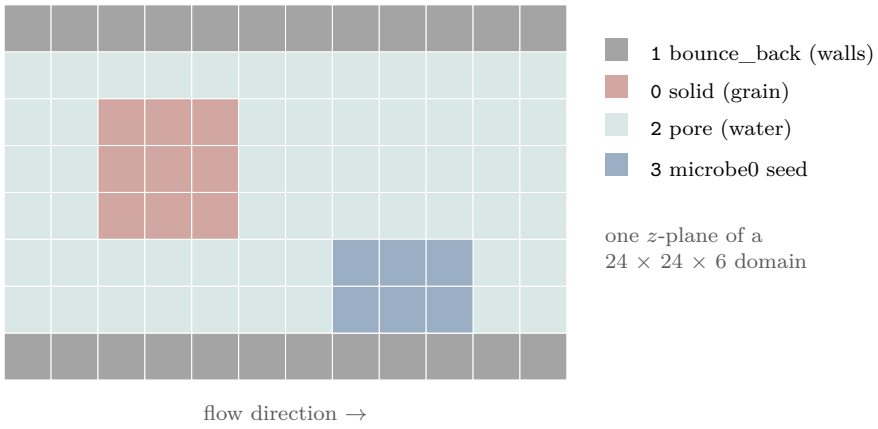
4.3 `<LB_numerics>`: domain and time

```
<domain>
    <nx>24</nx> <ny>24</ny> <nz>6</nz>
    <dx>10</dx> <unit>um</unit>
    <characteristic_length>24</characteristic_length>
    <filename>geometry.dat</filename>
    <material_numbers>
        <pore>2</pore>
        <solid>0</solid>
        <bounce_back>1</bounce_back>
        <microbe0>3</microbe0>
    </material_numbers>
</domain>
```

`<dx>` and `<unit>` set the physical size of one voxel; here 10 µm. `<characteristic_length>` is the length scale the Reynolds and Péclet numbers are built on, in voxels.

4.3.1 Material numbers

Every voxel in the geometry file carries an integer. The `<material_numbers>` block says what those integers mean.



The distinction between `solid` and `bounce_back` matters: both stop the flow, but `solid` voxels get no dynamics at all, while `bounce_back` voxels reflect the fluid. Use `bounce_back` for the domain walls and `solid` for grains.

A `<microbeN>` entry does double duty: it says which voxels that organism is seeded in, *and* it marks the organism as a biofilm former rather than a planktonic one. That has consequences described in Section 4.5.1.

4.3.2 The geometry file

The geometry is a plain text file, one integer per line, x fastest, then y, then z:

```
for z in range(nz):
    for y in range(ny):
        for x in range(nx):
            write(mask[x][y][z])
```

So the file has exactly `nx*ny*nz` lines. There is no header. The examples ship a generator, `examples/makeGeometry.py`, which is the easiest thing to adapt for your own geometry.

> **Order, not shape.** Nothing in the file records `nx`, `ny` or `nz` — those come from the XML. A file with the right number of values but the wrong loop order produces a geometry that is silently transposed. If your pore structure looks scrambled in ParaView, this is why.

4.4 <chemistry>: the solutes

One `<substrateN>` block per solute, numbered from zero with no gaps.

```
<chemistry>
    <number_of_substrates>2</number_of_substrates>

    <substrate0>
        <name_of_substrates>donor</name_of_substrates>
        <initial_concentration>0.1</initial_concentration>
        <substrate_diffusion_coefficients>
            <in_pore>5e-10</in_pore>
            <in_biofilm>5e-10</in_biofilm>
        </substrate_diffusion_coefficients>
        <left_boundary_type>Dirichlet</left_boundary_type>
        <left_boundary_condition>0.5</left_boundary_condition>
        <right_boundary_type>Neumann</right_boundary_type>
        <right_boundary_condition>0.</right_boundary_condition>
    </substrate0>
    ...
</chemistry>
```

Dirichlet holds the concentration at the boundary; **Neumann** sets the flux, and `0` means no flux, i.e. a closed end.

Two optional flags are worth knowing early:

`<immobile>`	`true` means this substrate never advects or diffuses. It only receives its reaction source term. This is how minerals are represented: they must accumulate exactly where they form.
`<fix_concentration>`	a per-substrate list in `<chemistry>`. Marks a solute as an inexhaustible reservoir: reactions read it but never draw it down.

4.5 <microbiology>: the organisms

One `<microbeN>` block per organism. Two of its tags carry most of the meaning, and they are independent of each other.

```
<microbe0>
    <name_of_microbes>Bug</name_of_microbes>
    <solver_type>CA</solver_type>          <!-- how biomass MOVES -->
    <reaction_type>kinetics</reaction_type><!-- how it METABOLISES -->
    <initial_densities>1.0</initial_densities>
    <decay_coefficient>0.0</decay_coefficient>
    <viscosity_ratio_in_biofilm>0</viscosity_ratio_in_biofilm>
    <half_saturation_constants>0.05 0</half_saturation_constants>
    ...
</microbe0>
```

4.5.1 Biofilm or planktonic

An organism with a `<microbeN>` entry under `<material_numbers>` is a *biofilm* former: it is seeded in those voxels and it can clog them. One without is *planktonic*: seeded uniformly through the pore space.

Three rules follow, and each stops a run when broken:

- `<viscosity_ratio_in_biofilm>` must be present for exactly the biofilm organisms. The parser counts them and compares.
- `<initial_densities>` must have one entry per material number the organism is seeded in.
- `solver_type FD` is rejected for planktonic organisms.

4.6 Where to look things up

Question	Where
what does this tag do, exactly?	Chapter 23, or the inline comments in `CompLaB_reference_template.xml`
what is a realistic value?	the example closest to your problem, Part II
why did the run refuse to start?	Chapter 24

> `CompLaB_reference_template.xml` ships with the code and documents every tag inline, next to a plausible value, with its units and known limits. It is the fastest reference when you are actually editing a file.

Part II

Tutorial

Each chapter here is built on a worked example that ships with the code, in `examples/`. They are tiny — $24 \times 24 \times 6$ voxels, seconds on a laptop — because the point is to show what a switch does, not to be physically interesting.

Every chapter has the same shape: what the case is, what to run, what you should see, and what to change to learn something. Four of the cases have an answer that does not come from this code, and those are flagged: they are the ones worth trusting.

CHAPTER 5

Transport with no reaction

Examples 01 and 02.

5.1 The point

Before adding chemistry, be sure transport is right. These two cases have no reactions at all, and the second has an answer you can check on paper.

5.2 Example 01: flow and a tracer

A tracer is held at 1 on the inlet face and 0 on the outlet, and reacts with nothing. `<Peclet>` is 1, so advection and diffusion are comparable.

Expect: the tracer sweeping left to right; a profile that falls monotonically with x and never rises; no `[NEG!]` warnings; no dependence on z.

5.3 Example 02: diffusion only

The same case with `<Peclet>` set to 0, which switches the flow solver off entirely.

> **This one has a right answer.** With no flow, no reaction, one face held at 1 and the other at 0, the steady state is the solution of $\nabla^2 C = 0$ with those boundaries — a **straight line** from 1 to 0 along x.
> Plot the centreline profile. If it is straight, your diffusion coefficient, your boundary conditions and your relaxation time are all doing what you think. If it curves, one of them is not, and it is far easier to find out here than in a case with biology in it.

5.4 What to change

Change	What it tells you
`<Peclet>` 0 → 1 → 10	how the balance of advection and diffusion moves the front. At high Péclet the front sharpens and the transport solver needs smaller steps.
`<tau>` away from 0.8	the lattice-Boltzmann relaxation time. Values far from 1 are less stable; below about 0.55 the scheme misbehaves.
right boundary Dirichlet → Neumann	a closed end: the tracer accumulates instead of leaving.

If example 02 is not straight, stop here. Every later chapter builds on transport being correct. A curved profile with no reaction means something is firing that should not be, or a boundary is not being held, and both will corrupt everything downstream while looking plausible.

CHAPTER 6

Abiotic kinetics

Example 03.

6.1 The point

Chemistry with no biology. Two reactants enter from opposite faces, meet, and make a product:

$$\mathrm{A} + \mathrm{B} \longrightarrow \mathrm{C}, \qquad \text{rate} = k\,[\mathrm{A}][\mathrm{B}]$$

6.2 Where the rate law lives

In `defineAbioticKinetics.hh`, in the example's own folder. The whole function is this:

```
void defineAbioticRxnKinetics(std::vector<double> C,
                              std::vector<double>& subsR,
                              plb::plint mask)
{
    for (std::size_t i = 0; i < subsR.size(); ++i) subsR[i] = 0.0;
    if (C.size() < 3 || subsR.size() < 3) return;
    if (mask < 2) return;              // nothing happens in solid or wall voxels

    const double A = std::max(C[0], 0.0);
    const double B = std::max(C[1], 0.0);
    const double R = k_AB * A * B;     // mol/L/s

    subsR[0] = -R;
    subsR[1] = -R;
    subsR[2] = +R;
}
```

Three things in that snippet are conventions you will reuse everywhere:

- **`C` is indexed in `CompLaB.xml` order.** `C[0]` is `<substrate0>`. There is no lookup by name.
- **`subsR` is a rate, per second**, positive for produced and negative for consumed. The solver multiplies by the time step.
- **`std::max(x, 0.0)`** floors the tiny negative values the lattice-Boltzmann solver can produce near steep fronts. Without it a rate law can amplify numerical noise into a real instability.

6.3 What you should see

A peak in C near the middle, with A and B drawn down around it. With `<Peclet>` at 0 the front is stationary; turn the flow on and it moves downstream.

This one has a right answer too. One A plus one B makes exactly one C, so summed over the domain

$$\Delta[\mathrm{A}] + \Delta[\mathrm{C}] = 0 \qquad \text{and} \qquad \Delta[\mathrm{B}] + \Delta[\mathrm{C}] = 0$$

at every step. Any drift in those sums is a bug, not chemistry.

CHAPTER 7

Aqueous speciation

Example 04.

7.1 The point

Some chemistry is fast enough to treat as instantaneous equilibrium rather than as a rate. CompLB3D solves a components-and-species tableau in every pore voxel, at every step.

The example uses the smallest tableau that is still real chemistry: the carbonate system, four species over two components.

species	HCO_3	H	$\log K$
HCO_3 *(component)*	1	0	0
H *(component)*	0	1	0
CO_3	1	−1	−10.33
H_2CO_3	1	1	6.35

In XML:

```
<equilibrium>
    <enabled>true</enabled>
    <components>HCO3 H</components>
    <stoichiometry>
        <species0>1  0</species0>
        <species1>0  1</species1>
        <species2>1 -1</species2>
        <species3>1  1</species3>
    </stoichiometry>
    <logK>
        <species0>0.</species0>     <species1>0.</species1>
        <species2>-10.33</species2> <species3>6.35</species3>
    </logK>
</equilibrium>
```

7.2 The rules of the tableau

- `<components>` names master species, and those names must match `<name_of_substrates>` exactly.
- Each `<speciesN>` row has one number per component, and `N` is the substrate index.
- A component's own row is the identity row with $\log K$ of 0.
- **A row of zeros means the solver ignores that species.** That is how minerals and biomass stay out of the tableau while still being substrates.

7.3 What you should see

The banner reports the tableau it parsed — 2 components, 4 species. Acid enters from the left, so H_2CO_3 should dominate near the inlet and CO_3 away from it.

Check the totals. Total carbonate, summed over HCO_3, CO_3 and H_2CO_3, must change only through transport. Speciation moves mass *between* species; it never creates or destroys a component.

This is the expensive part. Speciation costs more than everything else in the code combined. Even on 3456 voxels you will see it in the step time. Leave it off until you need it, and if you do need it, expect the equilibrium solve to dominate your budget.

7.4 Known limits, stated plainly

Activities are ideal — no Debye–Hückel or Davies correction. There is no temperature dependence, no gas phase, no redox couple and no mineral saturation index. The $\log K$ values you supply are therefore conditional constants at your own ionic strength and 25 °C. Say so in your methods section.

CHAPTER 8

Biomass and the three solvers

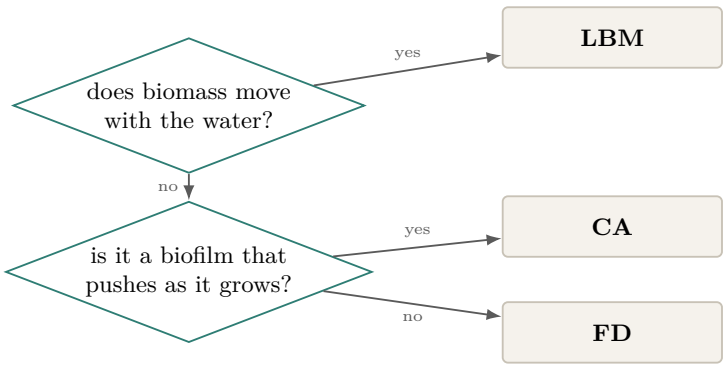
Examples 05, 06 and 07 — the same case, one line different.

8.1 The point

Biomass has to go somewhere when a colony grows. CompLB3D offers three ways of moving it, chosen per organism with `<solver_type>`. Examples 05 to 07 are identical apart from that one tag, so running all three shows you exactly what the choice costs.

8.2 The three

	How it moves biomass	Use it when
CA	cellular automaton: biomass over the maximum density spills into neighbouring voxels	you want a biofilm that spreads by pushing, which is how biofilms actually grow
FD	finite difference: biomass diffuses with its own coefficient	you want smooth spreading and a diffusivity you can justify
LBM	biomass gets its own advection–diffusion lattice	biomass genuinely moves with the flow



8.3 What each one demands of the input file

CA	needs `<viscosity_ratio_in_biofilm>`, `<thrd_biofilm_fraction>` and `<CA_method>`. `<CA_method> fraction` shares the excess in proportion to the space free in each neighbour.
FD	needs `<biomass_diffusion_coefficients>`. **Rejected for planktonic organisms** — it requires a `<microbeN>` entry under `<material_numbers>`.
LBM	needs `<biomass_diffusion_coefficients>`. Costs one extra lattice per organism, which is the most expensive of the three in memory.

8.4 The rate law

`defineKinetics.hh` in these folders implements Monod growth:

$$\text{growth} = \mu_{\max} \frac{S}{K_s + S} B, \qquad \frac{dS}{dt} = -\frac{\text{growth}}{Y}, \qquad \frac{dB}{dt} = \text{growth} - k_d B$$

> **`<half_saturation_constants>` in the XML does not drive this.** That tag feeds the flux balance and surrogate paths, which build their own uptake bounds from it. A hand-written kinetics file takes its parameters from its own table, in the header, next to the equation that uses them. If you change K_s in the XML and nothing happens, this is why.

8.5 What you should see

Biomass rising from the seeded patch and then levelling off as growth and decay balance. The donor drawn down around the colony and replenished from the boundary. A product plume spreading from the colony. No `[NEG!]` warnings.

Run 05, 06 and 07 and compare: the biomass *totals* should be close, the spatial *patterns* different. That is the honest way to see what the solver choice actually buys you.

CHAPTER 9

More than one organism

Example 08.

9.1 The point

`<solver_type>` and `<reaction_type>` are per organism, not global. Example 08 has two microbes on opposite walls sharing one donor, one using the cellular automaton and the other lattice Boltzmann, in the same run.

They differ in kinetics too: Bug1 has the higher $\mu_{\max}$ and the higher K_s; Bug2 the reverse. So Bug1 wins where the donor is plentiful and Bug2 where it is scarce — the classic r versus K trade-off. Bug1 sits nearer the inlet, so it should end up ahead.

9.2 What you should see

Both populations growing, then their *ratio* going flat. A donor shadow between the two colonies.

9.3 What to change

Swap the two `<solver_type>` lines and rerun. The converged ratio should barely move: the solver decides how biomass spreads, not how much of it there is. If it moves a lot, that is worth understanding before you trust either solver on a real problem.

CHAPTER 10

Flux balance analysis

Examples 09 and 10.

10.1 The point

Instead of writing a rate law, hand the organism a genome-scale metabolic model and let a linear program work out how fast it can grow on what is locally available. CompLB3D solves that program in every voxel, at every step.

10.2 The toy model, solved by hand

Example 09 uses a model small enough to check on paper — three metabolites, four reactions:

index	reaction	written as	meaning
0	EX_S	$S_c \rightarrow$	negative flux imports the donor
1	EX_O	$O_c \rightarrow$	negative flux imports the acceptor
2	EX_P	$P_c \rightarrow$	positive flux exports the product
3	BIO	$S_c + 0.5\,O_c \rightarrow P_c$	the objective

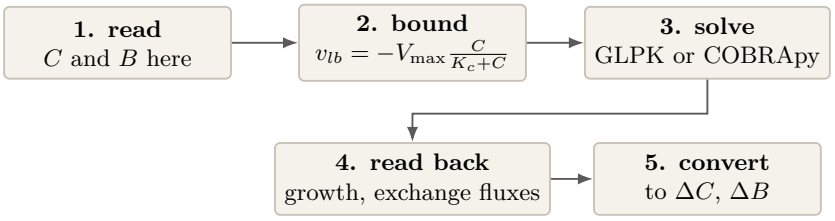
Steady state forces $v_0 = -v_3$ and $v_1 = -\frac{1}{2}v_3$, so with uptake bounds V_S and V_O:

$$\boxed{\text{growth} \;=\; \min(V_S,\; 2\,V_O)} \qquad \text{where} \qquad V = V_{\max}\,\frac{C}{K_c + C}$$

With `fba_maximum_uptake_flux` of 10 and 4, and both concentrations well above their half-saturation constants, $V_S \rightarrow 10$ and $2V_O \rightarrow 8$: the acceptor limits, and growth should settle near **8 mmol/gDW/h**.
That number comes from the stoichiometry, not from this code. It is the strongest check in the suite.

Change `<fba_maximum_uptake_flux>` to `4. 10. 0.` and the donor limits instead, so growth should settle near 4. One line, a second independent check.

10.3 How a voxel gets its answer



Steps 1, 2, 4 and 5 are identical for GLPK and COBRApy. Only step 3 differs.

10.4 GLPK or COBRApy

	GLPK	**COBRApy**
how it works	a persistent `glp_prob`; per voxel it only resets column bounds and re-solves from the previous basis	rebuilds the problem through optlang on every `optimize()` call, across a Python boundary
speed	the fast path	one to two orders of magnitude slower per voxel
needs	`-DENABLE_GLPK=ON`	`-DENABLE_COBRAPY=ON`, cobrapy installed, and `src/complab3d_cobrapy.py` present at run time

COBRApy does not read SBML. Both solvers eat the same `*_rGEM_c.xml` matrix file produced by `extractMM.py`. COBRApy rebuilds a `cobra.Model` in memory from that matrix, with anonymous numbered reactions — no gene names, no annotations. Its value here is as an independent second opinion, not as a richer interface.

10.5 Getting your own model in

`extractMM.py` converts a genome-scale model into the matrix file the solvers read.

```
python3 extractMM.py iAF987.xml -o input/geobacter.xml
python3 extractMM.py iJO1366.mat -o input/ecoli.xml     # COBRA Toolbox .mat
python3 extractMM.py model.json  -o input/model.xml
```

Then in `CompLaB.xml`:

```
<model_filename>geobacter</model_filename>          <!-- no .xml -->
<exchange_reaction_indices>6 55 418 -1</exchange_reaction_indices>
<fba_maximum_uptake_flux>15. 25. 8. 0.</fba_maximum_uptake_flux>
<substrate_lower_bounds>-15. -25. -8. 0.</substrate_lower_bounds>
<substrate_upper_bounds>60. 60. 60. 0.</substrate_upper_bounds>
<objective_direction>maximize</objective_direction>
<biomass_molar_mass>24.6</biomass_molar_mass>
```

`<exchange_reaction_indices>` maps each substrate, in XML order, onto the reaction in *that organism's* model that exchanges it. Use `-1` for a substrate the organism does not exchange — a

single `-1` is what makes a mutualism obligate.

> `<biomass_molar_mass>` converts CompLB3D's molar biomass into the gDW a metabolic model expects; 24.6 g/mol is the generic $CH_{1.8}O_{0.5}N_{0.2}$ formula weight. If your biomass is already in gDW/L, set it to 1.

10.6 What you should see

The model loading with its metabolite and reaction counts. The metabolic summary naming the solver and the number of microbes using it. Growth approaching the value the stoichiometry predicts. Product accumulating at the rate biomass is made.

Run examples 09 and 10 and compare. They receive an identical linear program and share nothing else, so agreement is strong evidence the coupling is right and disagreement localises the bug to one of the two.

CHAPTER 11

Surrogate models

Example 11.

11.1 The point

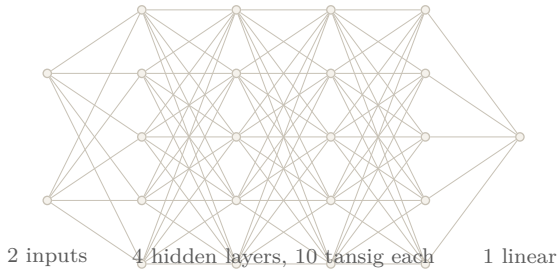
Flux balance analysis is expensive: one linear program per organism, per active voxel, per step. On a domain of a few million voxels it dominates everything.

A surrogate replaces the program with a small function fitted to it offline:

$$\text{uptake rates} \longrightarrow \text{growth rate}$$

evaluated in a few hundred multiplications. Speed-ups of 100 to 1000 times are typical.

11.2 The shipped network



The network shipped in `surrogateModel.hh` is *Geobacter metallireducens* on acetate and Fe(III), exported from MATLAB's Deep Learning Toolbox.

A network is only valid inside the range it was trained on. Outside it, a neural network does not fail — it returns confident nonsense. The shipped one was trained on

acetate	0.00089	to	9.998	mmol/gDW/h
Fe(III)	0.000035	to	0.4997	mmol/gDW/h
growth	0	to	0.0527	1/h

Ask it for an acetate uptake of 50 and an Fe(III) uptake of 5, well outside the box, and it cheerfully reports 0.0486 1/h.
That is why example 11 sets `<fba_maximum_uptake_flux>` to `8. 0.4` — safely inside. Raise the second number above 0.5 and the run spends its time extrapolating.

Those bounds are not documented anywhere in the network file. They were recovered by inverting the `mapminmax` scaling, which is what `surrogate_training/inspectSurrogate.py` does:

```
python3 surrogate_training/inspectSurrogate.py surrogateModel.hh
```

Run that on any network before trusting it, including one somebody else fitted.

11.3 Fitting your own

The shipped network encodes one organism on two substrates over one range. For anything else you need to train your own, and Chapter 20 covers that end to end.

CHAPTER 12

Combining metabolism types

Example 12.

12.1 The point

A metabolic model describes what the network can do; it does not describe everything a cell does. Maintenance is the standard example: a constant per biomass drain that yields no growth. A plain flux balance objective omits it, so the model over-predicts growth at low substrate.

The combined reaction types run both paths and **add** the rates:

`glpk_and_kinetics` `cobrapy_and_kinetics` `surrogate_and_kinetics`

12.2 The sign discipline

> **Do not write `bioR` in a kinetics file that runs alongside FBA.** Growth belongs to the metabolic path. Writing it in both places double counts it, and the result looks plausible — just wrong by a factor that depends on the substrate field.
> Example 12's `defineKinetics.hh` writes `subsR` only, and says so in a comment. Copy that discipline.

12.3 Two Vmax tags, not one

A combined organism needs both:

`<maximum_uptake_flux>`	read by the **kinetics** path, in mol substrate per mol cells per second
`<fba_maximum_uptake_flux>`	read by the **metabolic** path, in mmol/gDW/h

One number cannot satisfy both meanings. If you supply only the shared tag, the metabolic layer uses it and prints a warning about the units.

12.4 What you should see

The same growth rate as example 09, because maintenance adds no biomass — but the donor drawn down faster, by exactly the maintenance term. Diff the two donor profiles: the difference *is* the maintenance, and it confirms the paths are adding rather than one silently winning.

CHAPTER 13

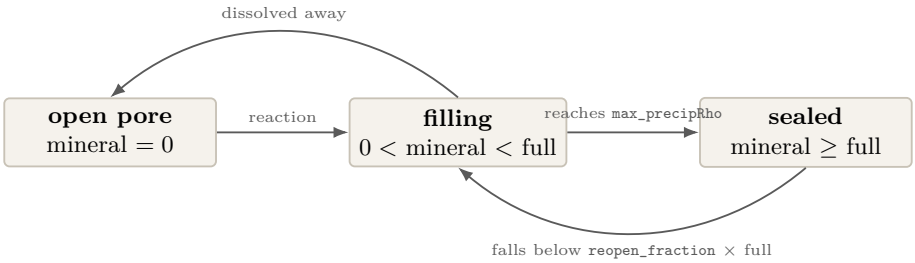
Precipitation and dissolution

Examples 13, 14 and 15.

13.1 The point

Minerals form, fill pore space and block the flow; acid arrives and eats them back out again. The geometry is not a fixed input — it is a state variable.

13.2 The voxel life cycle



13.3 Example 13: precipitation

$$Fe^{2+} + HS^- \longrightarrow FeS(s)$$

Three things make this work:

1. The mineral substrate is **<immobile>**. It never advects or diffuses, so it accumulates exactly where it forms. A mineral that moved would never seal anything.
2. **<max_precipRho>** is the mol/L of a completely full voxel, derived from the mineral: $1000/V_m$ with V_m in cm^3/mol. Mackinawite is 87.91 g/mol over 4.30 g/cm^3, so $V_m = 20.44$ and **<max_precipRho>** = 48.9.
3. **<surface_only>** 1 restricts growth to wall-adjacent voxels — heterogeneous nucleation, which is the physically right choice. Set it to 0 and the mineral grows anywhere, sealing pore throats far too readily.

Expect: FeS building in a band across the middle; the porosity in the log falling in steps, one per update interval; the flow re-solved after each conversion; and mass conserved — every mole of FeS removes one Fe^{2+} and one HS^-.

13.4 Example 14: dissolution

$$CaCO_3(s) + H^+ \longrightarrow Ca^{2+} + HCO_3^-$$

Dissolution has its own hook, `defineDissolutionRate()`, in the same header. It is handed:

`phaseId`	which declared phase this is, numbered from 1 in `<phaseN>` order
`C`	the water **touching** the mineral: the open neighbours' concentrations, averaged. A solid voxel has no water of its own that moves.
`mineral`	mol/L of mineral left in this voxel

and must return `mineralR` **negative**, plus the released solutes in `subsR`, positive. Do not write the mineral's own `subsR` entry — the solver does that from `mineralR`.

13.4.1 Why phases must be declared

Only what appears in a `<phaseN>` block can dissolve, and that is deliberate. If the rule were simply "a solid voxel with no mineral left becomes pore", then an inert quartz grain — which never held any mineral — is already below any threshold, and your whole grain pack would dissolve on the first step. Declaring phases makes quartz safe by construction.

```
<phase0>
    <name>calcite_grain</name>
    <material_number>0</material_number>   <!-- the solid grains -->
    <substrate>2</substrate>               <!-- holds its inventory -->
    <full_density>27.1</full_density>      <!-- mol/L when full -->
    <initial_fill>27.1</initial_fill>      <!-- starts solid -->
</phase0>
```

`<initial_fill>` is the point of example 14: the grains *start* solid and are eaten away. Dissolution is not limited to material that precipitated first. A precipitate phase instead uses `<initial_fill>` 0 and `<is_precipitate> true`.

Expect: the acid eating into the *upstream* faces first; Ca^{2+} leaving downstream; porosity rising in steps, the mirror of example 13.

13.5 Example 15: both at once

Both blocks enabled, on one mineral, so a voxel can seal and later reopen.

> **This is what `<reopen_fraction>` is for.** With both directions active a voxel can sit near the threshold. A single threshold makes it flip open and shut every update interval, and every flip re-solves the flow. `<reopen_fraction>` 0.9 puts a gap between sealing at full density and reopening below 0.9× full, so it settles instead of chattering.
> Set it to 0.99 and rerun: you should see the porosity oscillate, with a flow solve wasted on each flip. That is the failure the hysteresis prevents.

The two rate laws are *not* inverses. Precipitation is second order in the dissolved ions; dissolution is first order in acid and in remaining mineral. They are separate laws that happen to move the same mineral — which is how the code has to work, because the speciation solver computes no solubility products and so there is no saturation state to drive a single reversible rate.

Part III

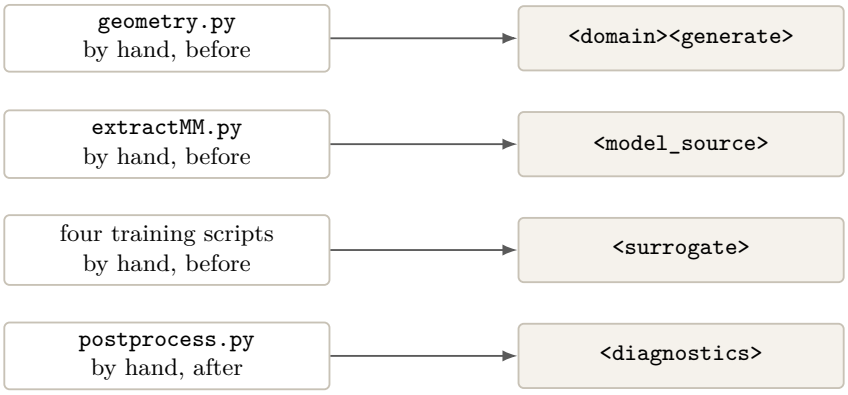
The pipeline blocks

CHAPTER 14

What these four blocks replace

Everything up to here has assumed that the files a run needs already exist: a `geometry.dat` somebody made, a metabolic model somebody converted, a surrogate somebody trained, and a habit of loading the output into Python afterwards to find out whether anything was conserved.

Those four assumptions were four scripts, run by hand, in the right order, before and after every simulation. This part is about the four XML blocks that do them instead.



The old way still works. None of these blocks is required, and every one of them is absent by default. An input file written before they existed takes exactly the path it always took — the code checks for the block, does not find it, and carries on. The Python tools in `tools/` are still shipped and still supported; they do things C++ cannot, such as reading a TIFF stack or drawing a plot.

One thing genuinely changed. `<model_filename>` may now point at an SBML file directly, and `extractMM.py` is no longer needed for the GLPK path. The old matrix files still load — the format is decided by looking at the file's root element, not its name — so nothing you already have stops working.

CHAPTER 15

Where your metabolic model comes from

15.1 The point

A genome-scale model is a published artefact with a revision. Chapter 23 treats `<model_filename>` as just a path, but in practice getting that path right was three manual steps: find the model on BiGG, download it, run `extractMM.py` to flatten it into the matrix format CompLaB used to read. Each step is somewhere the wrong file can be substituted without anything complaining.

15.2 Reading SBML directly

The simplest change is that `<model_filename>` can now name the SBML file itself:

```
<model_filename>iJO1366.xml</model_filename>
```

The reader is `complab3d_sbml.hh`. It produces the same S, b, c, lb, ub and objective column that `extractMM.py` did, in the same order, and was checked against `cobrapy` on *iJO1366*: identical reaction order, identical 4.7 million-entry stoichiometry matrix, identical bounds, identical objective. It takes about 0.35 s where `cobrapy` takes 5 s, because it does one pass of tinyxml and no object graph.

> **How the format is decided.** By the root element. `<sbml>` means SBML; `<Metabolic_Model>` means the `extractMM.py` matrix format. The file extension is never consulted, so renaming a file cannot change how it is read.

15.3 Naming exchange reactions instead of numbering them

This is the part worth changing in an existing input file even if you change nothing else.

```
<!-- before: column numbers, correct only for one revision of one model -->
<exchange_reaction_indices>27 35 -1</exchange_reaction_indices>

<!-- after: names, checked against the model at start-up -->
<exchange_reaction_names>EX_glc__D_e EX_o2_e none</exchange_reaction_names>
```

Both mean the same thing — which reaction of the metabolic model exchanges each substrate, in `<name_of_substrates>` order. The difference is what happens when they are wrong.

- An index that points at the wrong reaction is not detectable. The run proceeds, the linear program is well posed, and the answer is wrong everywhere.
- A name that does not exist stops the run at start-up, names the substrate, and lists the boundary reactions whose names are close.

Use `none` (or `-1`) for a substrate this organism does not exchange. If both tags are given, the names win and a note says so.

> **Names need SBML.** The `extractMM.py` matrix format does not carry reaction names, so `<exchange_reaction_names>` with a matrix file is a hard error rather than a silent fallback. Point `<model_filename>` at the SBML.

15.4 Letting the run find the model

```
<model_source>bigg:e_coli_core</model_source>
<model_bundle>models</model_bundle>      <!-- shipped with the code -->
<model_cache>input</model_cache>         <!-- where the unpacked file goes -->
<allow_download>false</allow_download>   <!-- default: never touch the network -->
```

The search order is fixed and is reported in the log:

1. **already unpacked** in `<model_cache>`. Used as is.
2. **bundled** with the code as `models/<id>.xml.gz`. Unpacked into the cache. Three models ship this way: *e_coli_core*, *iJO1366* and *STM_v1_0*.
3. **downloaded**, and only if `<allow_download>` is `true`. A cluster compute node usually has no route to the internet, which is exactly why the default is `false` and why the models are bundled.

Whichever branch runs, the file is checked against `models/manifest.txt`, which records for each model its species count, reaction count, parsed metabolite and reaction counts, objective reaction, SHA-style hash, byte size and source URL. A mismatch does not stop the run; it prints what it expected and what it found, so a model that has been revised upstream is visible in the log rather than buried in the results.

> **One microbe, or all of them.** The resolved file is given to every metabolic microbe that did not name its own `<model_filename>`. A microbe with an explicit one keeps it, so a two-organism run can mix a bundled model with a local one.

> **Parallel runs.** Only rank 0 unpacks or downloads. The others wait at a barrier and then find the file already there. Nothing is written twice and no two ranks race for the same path.

CHAPTER 16

Making a pore space

16.1 The point

`tools/geometry.py` can build a packing or threshold an image stack, and it is still the right tool when the source is a TIFF stack or a PNG series. But for the common cases — a sphere pack at a given porosity, a fracture, a layered medium — there is no reason to leave the input file.

16.2 Generating

Inside `<domain>`, alongside `<nx>` and friends:

```
<generate>spheres</generate>      <!-- channel | spheres | cylinders |
                                       random | fracture | layered -->
<porosity>0.55</porosity>
<grain_radius>3.0</grain_radius>
<seed>12345</seed>
<walls>y</walls>                  <!-- solid walls on the named axes -->
<write_geometry>generated.dat</write_geometry>
```

The generated volume is written to `<write_geometry>` inside the input directory before the run starts. That file is the record: the run is reproducible from its own output even if the seed, the code or the XML later change, and the generated geometry can be inspected with exactly the same tools as one that came from imaging.

16.3 Importing a binary volume

```
<import_raw>rock.raw</import_raw>
<raw_dtype>uint8</raw_dtype>
<threshold>128</threshold>
<invert>false</invert>
```

A flat binary volume, thresholded into pore and solid. `<generate>` and `<import_raw>` are mutually exclusive and setting both stops the run.

16.4 What the inspection tells you

Either way, the pore space is inspected before the flow solver is built, and the report is printed:

```
[GEOM] generated a 'spheres' pore space
[GEOM] 24 x 24 x 6 = 3456 voxels
[GEOM]   material 1   bounce_back    288    8.33%
[GEOM]   material 0   solid         1170   33.85%
[GEOM]   material 2   pore          1998   57.81%
[GEOM] porosity 0.5781
[GEOM] percolates along x: yes, 1 spanning cluster(s)
```

```
[GEOM] isolated pore space: 1 voxel(s), 0.1% of the open space
[GEOM] written to input/generated.dat so this run can be reproduced exactly
```

A sealed domain stops the run. If the pore space does not percolate along x, the run stops before the flow solver starts. A pressure drop across a sealed medium has no solution, and the flow solver would spend its entire iteration budget failing to converge to one. Lower the porosity target, or shrink the grains.

Isolated pockets are reported, not removed. Pore voxels that no percolating path reaches still hold substrate and still host biology, and nothing about them is wrong — but they never exchange with the through-flow, so a mass balance that includes them behaves differently from one that does not. The count is printed so the number is not a surprise.

The file is `nx` slices wide, where `nx` is what you wrote in the XML. Internally the code adds two ghost columns, and `readGeometry` duplicates the first and last slice into them. The generator is given the width you asked for, so this is handled — but it is why a geometry file made by hand must have exactly `nx` × `ny` × `nz` entries and not two slices more.

CHAPTER 17

A surrogate without leaving the XML

17.1 The point

Chapter 20 describes the offline route: sweep the FBA into a CSV, fit a network, export it as a header, paste the header into `surrogateModel.hh`, rebuild. That route still exists and is still the right one when you want to inspect the fit, try several architectures, or train on a machine that is not the one you will run on.

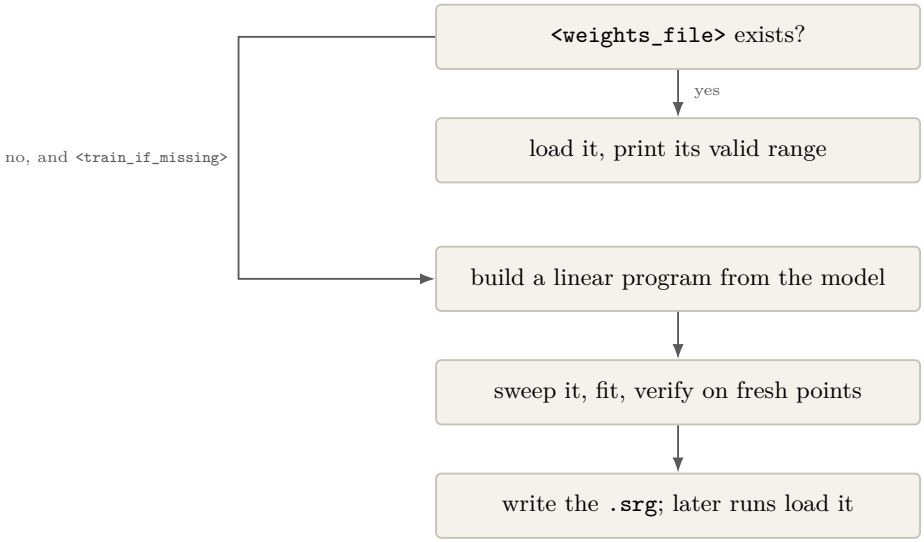
The change is that the weights now live in a **file the solver reads at run time** rather than in a header it was compiled with. Once that is true, the run can also fit the network itself.

17.2 The block

```
<surrogate>
    <enabled>true</enabled>
    <weights_file>output/ecoli.srg</weights_file>
    <train_if_missing>true</train_if_missing>
    <train>
        <samples>2000</samples>
        <layers>10 10</layers>
        <restarts>5</restarts>
        <epochs>4000</epochs>
        <verify_points>512</verify_points>
        <seed>20260814</seed>
        <input0>S 0.1 10.0</input0>
        <input1>O 0.1 20.0 log</input1>
    </train>
</surrogate>
```

Each `<inputN>` line reads: *substrate name*, *low*, *high*, and optionally `log` for a logarithmic sweep. The substrate name must be one of `<name_of_substrates>`.

17.3 What happens at start-up



The linear program the sweep solves is not a copy. If the run already has a GLPK microbe, its own persistent `glp_prob` is used — the same object, with the same `<constraint_indices>` overrides, that the simulation would have solved. If it does not (the usual case: one organism, reaction type `surrogate`), a temporary problem is built from that microbe's own model and released as soon as training finishes.

> **Positive uptake in, negative bound out.** Exchange reactions are written `metabolite ->`, so consuming is a negative flux. The sweep passes positive uptake rates in `mmol/gDW/h`, the same convention as `Fin` everywhere else, and the sign is applied in exactly one place. An infeasible solve is recorded as zero growth rather than an error, so the network learns where the feasible region ends instead of having a hole in its domain.

17.4 Training is on rank 0 only

Every rank fitting its own network from its own random start would give a different growth rate in each subdomain. Rank 0 trains, writes the file, everyone barriers, the others read it.

17.5 Which substrate is which

A `.srg` file records the substrate name of every input:

```
inputs 2
inputnames S O
trainmin 0.127038797 0.118747061
trainmax 9.983995766 19.978325403
```

At start-up those names are matched against `<name_of_substrates>`, and a name that is not there stops the run. Without this a network trained on (acetate, Fe^{3+}) could be fed them the other way round: every number plausible, every growth rate wrong, nothing to see in the log.

A file with no `inputnames` line. Networks exported by the older MATLAB route have no such line. They still load, and their inputs are assumed to be the first substrates in order — with a warning saying so. Check it.

17.6 Outside the training box

A neural network does not fail outside the range it was fitted on. It returns a confident number that means nothing. So the range travels with the weights, and at run time every evaluation is **clamped** to that box and **counted**:

```
[SRG] runtime network: 1076692 evaluation(s), 0 clamped (0.00%)
```

Clamping rather than extrapolating is the conservative choice: at the edge the network returns the growth rate at the nearest uptake that was actually solved for, which is a real answer for a nearby state. It is still an approximation, which is why the count is reported. Above 5% the report says, in as many words, that the results should not be used until the ranges are widened and the network refitted.

17.7 Does it agree with the FBA?

On *e_coli_core*, in a run otherwise identical, glucose and oxygen as the two inputs:

	GLPK, every voxel	**surrogate**
biomass change over the run	−4.3090%	−4.3107%
final product concentration	2.0871×10^{-2}	2.0871×10^{-2}
wall clock	492 s	3.3 s

A factor of about 150 in wall clock, for a difference in the answer of four parts in ten thousand. That ratio is the whole argument for surrogates, and it is why the training range matters so much: the speed is only worth having if the simulation stays inside the box.

CHAPTER 18

The run's own record

18.1 The point

Until this block existed, a CompLB3D run wrote VTI volumes and nothing else. Every scalar question — did mass balance, how did porosity change, when did a concentration first go negative — meant loading the volumes into Python afterwards and hoping the answer was still in them.

18.2 The block

```
<diagnostics>
    <enabled>true</enabled>
    <summary_csv>summary.csv</summary_csv>
    <interval>100</interval>            <!-- 0 = follow the VTI interval -->
    <tolerance>1e-6</tolerance>
    <conserve>Fe2 + Fe3</conserve>
    <conserve1>SO4 + HS</conserve1>
</diagnostics>
```

18.3 summary.csv

One row per interval, written next to the VTI files:

```
iteration,porosity,open_voxels,S_total,S_mean,S_min,S_max,O_total,...
0,0.578125,1998,1998,1,0,1
100,0.578125,1998,1832.4,0.917,0,1
```

Totals and means are over **open voxels only** — pore plus biofilm — and over the physical domain $x = 1 \dots n_x - 2$, excluding the two ghost columns. Both choices matter: including the solid would make the mean fall as biofilm grew for no chemical reason, and including the ghost columns would count the inlet and outlet slices twice, so a conserved total would jump whenever the inlet concentration changed.

18.4 The conservation checks

`<conserve>` names a sum of substrates that ought to be constant. The first recorded value is the baseline; after that the drift is

$$\text{drift} \;=\; \frac{|\Sigma_{\text{now}} - \Sigma_{\text{first}}|}{\max(|\Sigma_{\text{first}}|,\, |\Sigma_{\text{now}}|)}.$$

Normalising by the larger of the two, rather than by the baseline, is what keeps the number meaningful when a run starts from an empty domain — there the baseline is zero and dividing by it reports 10^{302}, which tells nobody anything.

What to conserve, and what not to. A substrate held at a fixed concentration on an inlet is being supplied from outside. Its total is not conserved and never will be, and asking for it to be produces a failure that is entirely your own. Conserve a *closed* sum instead: every species sharing one element, which is a conserved moiety of the reaction network. The failure message says this, in those words, because it is by far the most common cause.

A negative concentration is reported once, loudly, the first time it appears, with the substrate and the step. At the end of the run each check is reported as PASS, FAIL, or SKIPPED:

```
[DIAG] summary written to output/summary.csv
[DIAG] Fe2 + Fe3            worst relative drift 3.104e-09  PASS
[DIAG] SO4 + HS             worst relative drift 1.882e-02  FAIL
[DIAG] verdict: FAIL
```

SKIPPED is not PASS. A `<conserve>` naming a substrate that does not exist is reported as SKIPPED, never as PASS. A skipped check that looks like a pass is how a mass-balance guarantee quietly becomes worthless.

Something this found. Running example 02 with no-flux boundaries at both ends — a genuinely closed box — the tracer total rises about 2.3% over the first two hundred steps and is then flat to 10^{-5} for the rest of the run. That is a start-up transient of the Neumann boundary implementation, not an ongoing leak, and it is present with or without any of the new code. It is recorded here because it is exactly the kind of thing this block exists to surface, and because a tolerance below about 3×10^{-2} on a Neumann case will report it.

Part IV

Going further

CHAPTER 19

Writing your own kinetics

19.1 The thing to understand first

Rate laws are compiled in, not read from the input file. `defineKinetics.hh` and `defineAbioticKinetics.hh` are `#include`d into the executable. Changing your chemistry means editing a C++ header and rebuilding. Two problems with different chemistry need two binaries.

This is the single most surprising thing about the code, and it is why every example folder carries its own pair of headers and why `runAllExamples.sh` rebuilds between cases.

19.2 The two hooks

`defineRxnKinetics`	biotic. Gets the biomass vector as well as the concentrations, and writes growth rates. Runs when `<enable_kinetics>` is true, for organisms whose `<reaction_type>` includes kinetics.
`defineAbioticRxnKinetics`	abiotic. Concentrations only. Runs when `<enable_abiotic_kinetics>` is true.
`defineDissolutionRate`	the dissolution hook, in the abiotic file. Runs only when `<dissolution>` is enabled.

19.3 Anatomy of a biotic rate law

```
void defineRxnKinetics(
    std::vector<double>  B,      // IN : biomass here, gDW/L, one per microbe
    std::vector<double>  C,      // IN : concentrations here, XML order
    std::vector<double>& subsR,  // OUT: dC/dt per substrate, per SECOND
    std::vector<double>& bioR,   // OUT: dB/dt per microbe,   per SECOND
    plb::plint           mask)   // IN : material number of this voxel
{
    for (std::size_t i = 0; i < subsR.size(); ++i) subsR[i] = 0.0;
    for (std::size_t i = 0; i < bioR.size();  ++i) bioR[i]  = 0.0;
    if (C.size() < 2 || subsR.size() < 2) return;   // guard: your chemistry size
    if (mask < 2) return;                           // no biology in solid/wall

    const double S = std::max(C[0], 0.0);

    for (std::size_t m = 0; m < B.size(); ++m) {
        const double Bm = std::max(B[m], 0.0);
        if (Bm <= 0.0) continue;

        const double growth = mu_max[m] * S / (Ks[m] + S) * Bm;   // gDW/L/s

        bioR[m]  += growth - kd[m] * Bm;
```

```
        subsR[0] -= growth / Y[m];
        subsR[1] += growth * fP[m];
    }
}
```

19.4 The six rules

1. **Indices follow `CompLaB.xml`.** `C[0]` is `<substrate0>`; `B[0]` is `<microbe0>`. There is no lookup by name, so reordering substrates in the XML silently changes what your rate law means. Keep a comment naming each index at the top of the file.
2. **Everything is per second.** A rate constant published per day must be divided by 86400.

   > **This causes more dead runs than anything else in the code.** A rate 86400 times too large blows up on the first step; a rate 86400 times too small does nothing at all and looks like a configuration problem. Write the unit in a comment next to every constant.

3. **Zero the output arrays first.** They are reused between voxels. A rate law that returns early without zeroing leaks the previous voxel's answer into this one.
4. **Guard on size, and match your chemistry.** The `if (C.size() < 2)` line is not decoration. The shipped uranium network guards with `C.size() < 14`; drop that file into a two-substrate case and it reads past the end of the array. `validateExamples.py` checks this guard against the substrate count for exactly that reason.
5. **Floor negatives.** `std::max(x, 0.0)` on every concentration you read. The lattice-Boltzmann solver can produce small negative values near steep fronts, and a rate law that squares one amplifies noise into an instability.
6. **Never call `exit()`.** Under MPI it leaves the other ranks hanging and destroys the run. Print a warning and return a safe value; a voxel that grows at zero rate is visible in the output and recoverable.

19.5 Checking a rate law before you trust it

Mass balance is the cheapest real test, and you can do it without Palabos. Compile the header on its own against a stub, feed it random states, and check the conservation you know must hold:

```
// mini driver: A + B -> C must satisfy dA + dC = 0 and dB + dC = 0
for (int trial = 0; trial < 20000; ++trial) {
    std::vector<double> C = {rnd(), rnd(), rnd()}, R(3);
    defineAbioticRxnKinetics(C, R, 2);
    assert(std::fabs(R[0] + R[2]) < 1e-12);
    assert(std::fabs(R[1] + R[2]) < 1e-12);
}
```

`validateExamples.py` does the compile half of this for every example; the balance half is worth writing for your own chemistry, once.

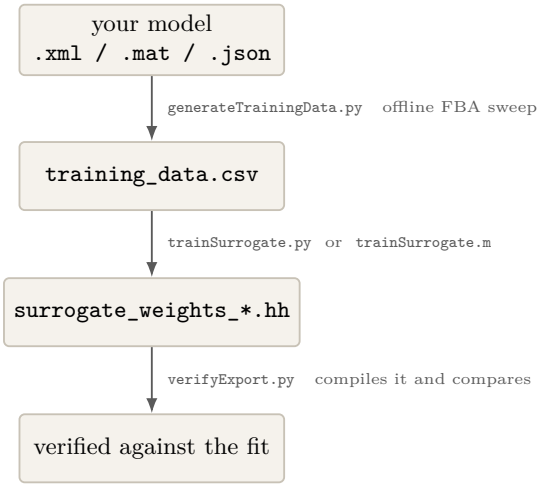
CHAPTER 20

Training your own surrogate

The network shipped in `surrogateModel.hh` encodes one organism, two substrates, one range. For anything else you fit your own.

> **Read chapter 17 first.** Since the `<surrogate>` block exists, the run can fit a network itself, from the XML, with no scripts and no rebuild. That is the shorter path and it is the one to try first. This chapter is the offline route, which is still the right one when you want to compare architectures, train on a different machine from the one you will run on, or keep the training data as a separate artefact. Everything for it is in `surrogate_training/`, and the two routes produce interchangeable results: `trainSurrogate.py` can write a `.srg` file the solver loads, and `inspectSurrogate.py` reads networks from either.

20.1 The three steps



20.2 Step 1: sweep the FBA offline

```
python3 generateTrainingData.py iAF987.xml \
    --exchange EX_ac_e   --range 1e-3 10   --log \
    --exchange EX_fe3_e  --range 1e-5 0.5  --log \
    --samples 4000 --seed 1 \
    -o geobacter_training.csv
```

20.2.1 Choosing the range

> Use the range the **simulation** will visit, not the range the organism can tolerate. CompLB3D's uptake bound is $V = V_{\max}C/(K_c + C)$, so the largest uptake any voxel can ever request is $V_{\max}$ — your `<fba_maximum_uptake_flux>`. That is the natural upper end of the sweep. The lower end should be small but non-zero.

Sample **logarithmically** whenever the range spans more than about one order of magnitude. Pore-network concentrations routinely span four, and a linear grid puts nearly all its points in the top decade — so the fit is worst exactly where the biology is most sensitive.

Keep to **1 to 3 inputs**. Beyond that the sample count needed for the same accuracy grows fast and the fit stops being worth the trouble.

20.3 Step 2: fit and export

Two paths, writing byte-identical output. Use whichever you have:

`trainSurrogate.m`	MATLAB, Deep Learning Toolbox, `trainlm`. Reconstructs how the shipped network was made.
`trainSurrogate.py`	numpy and scipy, L-BFGS-B on analytic gradients. No licence needed.

```
python3 trainSurrogate.py geobacter_training.csv \
    --name geobacter --microbe-id 0 \
    -o surrogate_weights_geobacter.hh
```

Both default to the shipped architecture — 4 hidden layers of 10 tansig neurons and a linear output, with `mapminmax` scaling on both ends — so a retrained network drops into an existing build with no other change.

20.4 Step 3: verify

```
python3 verifyExport.py surrogate_weights_geobacter.hh
```

> **Do not skip this.** An exporter that transposes a matrix or drops a digit produces a header that compiles, runs, and is wrong, with no independent reference to notice. So the trainer writes 512 reference predictions next to the header, and `verifyExport.py` compiles the header and checks them. It should report agreement to about 10^{-13} relative.

20.5 Installing it

> **The short way.** Write the fit as a `.srg` file and name it in `<weights_file>`. Nothing is compiled in, nothing is rebuilt, and the training range travels with the weights so the solver can refuse to extrapolate. The rest of this section is the older route, which bakes the weights into the executable.

The generated header is namespaced, so several organisms coexist:

```
#include "surrogate_weights_geobacter.hh"
...
else if (microbeId == 0) {
    bioR[microbeId] = srgnet_geobacter::eval(Fin[microbeId]);
}
```

Then `<enable_surrogate> true` and `<reaction_type> surrogate` in the XML.

Every generated header also carries `inTrainingRange()`. Call it if a run may wander outside the box.

20.6 Inspecting a network somebody else fitted

```
python3 inspectSurrogate.py surrogateModel.hh
```

The training range is not stored anywhere as such — but `mapminmax` is invertible, so it can be recovered exactly:

$$\text{gain} = \frac{2}{x_{\max} - x_{\min}}, \quad \text{offset} = x_{\min} \qquad \Longrightarrow \qquad x_{\max} = \text{offset} + \frac{2}{\text{gain}}$$

Run it before trusting any network, including the shipped one.

CHAPTER 21

Running on a cluster

21.1 MPI

MPI is on by default. Nothing in the input file changes; you launch with `srun` or `mpirun` instead of running the binary directly.

```
srun ./complab
```

Palabos decomposes the domain into blocks, one per rank, with a halo. That has a consequence worth internalising:

> **More ranks is not always faster.** Each rank needs enough interior voxels to be worth the halo exchange. The shipped examples are 3456 voxels; on 36 ranks that is about 96 voxels each, nearly all of them halo, and the run is slower than on one core. Ask for ranks in proportion to your domain, not to what the queue will give you.

21.2 A Slurm script

`comp.sh` is a production template; `examples/runExamples.slurm` submits the example suite. The parts that matter:

```
#SBATCH --account=PROJECT          # required, or the job is rejected
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=36       # match this to your domain size
#SBATCH --time=04:00:00

cd $SLURM_SUBMIT_DIR               # complab reads CompLaB.xml from the cwd
module purge
module load gcc openmpi            # THE SAME modules you built with
srun ./complab
```

> **Load the same modules you built with.** A binary built against one MPI and launched under another fails in ways that look like your code is broken. Put the `module load` line in both the build script and the run script, and keep them identical.

21.3 Optional dependencies on a cluster

GLPK	`module load glpk`, or build it in your home directory and point `CMakeLists.txt` at it
COBRApy	needs cobrapy importable **at run time**, not just at build time, and `src/complab3d_cobrapy.py` present at `<src_path>`. A virtualenv that is not activated in the job script is the usual failure.

21.4 Restarting

Long runs should checkpoint. Set `<save_CHK_interval>` to something non-zero, and resume with:

```
<read_NS_file>true</read_NS_file>
<read_ADE_file>true</read_ADE_file>
```

The flow field and the chemistry restart independently, which is useful: a converged flow field on a fixed geometry can be reused across many chemistry runs.

CHAPTER 22

Output and post-processing

22.1 What gets written

File	Format	Contents
`nsLattice*.vti`	VTK ImageData	velocity and pressure
`maskLattice*.vti`	VTK ImageData	material numbers, which change if minerals do
`subsLattice`n`*.vti`	VTK ImageData	substrate n
`bioLattice`n`*.vti`	VTK ImageData	microbe n
`*.chk`	Palabos binary	checkpoints, for restart

The index and the iteration number are appended to each filename, so `subsLattice0_0000500.vti` is substrate 0 at step 500.

22.2 ParaView

1. **Open** the numbered group; ParaView collapses the time series into one entry.
2. **Apply**, then set **Coloring** to the field.
3. **Slice** to see inside. The default view of a 3D block shows only its outer surface, which for a pore network is almost useless.
4. **Threshold** on the mask field to hide solid voxels.

> **Unit spacing.** The `.vti` files are written with spacing 1, so ParaView shows lattice coordinates. Multiply by `<dx>` for physical distance, or set the spacing in ParaView's Information panel.

22.3 Getting numbers out

For anything quantitative — a breakthrough curve, a mass balance, a converged biomass ratio — read the `.vti` files with a script rather than reading values off a picture.

```
# pip install vtk numpy
import vtk
from vtk.util.numpy_support import vtk_to_numpy

r = vtk.vtkXMLImageDataReader()
r.SetFileName("output/subsLattice0_0000500.vti")
r.Update()
img  = r.GetOutput()
```

```
dims = img.GetDimensions()
arr  = vtk_to_numpy(img.GetPointData().GetArray(0)).reshape(dims[::-1])

print("total:", arr.sum())          # for mass balance
print("mid-plane profile:", arr[dims[2]//2, dims[1]//2, :])
```

22.4 The checks worth automating

Check	Why
totals over time	mass balance. A conservative tracer's total should change only through the boundaries.
z-independence	every shipped example is quasi-2D; a z dependence is a bug.
negative minimum	should never be below zero by more than round-off.
porosity	for precipitation and dissolution runs, the number that tells the story.

Part V

Reference

CHAPTER 23

Every tag in CompLaB.xml

This chapter lists every element the code reads, with its type, default and units. Where a tag is mandatory only under some condition, that condition is stated: those are the ones that stop a run.

> **The file that documents itself.** `CompLaB_reference_template.xml` ships with the code and carries all of this as inline comments, next to a plausible value. When you are actually editing a file, that is faster than this chapter.

23.1 <path>

Tag	Type	Default	Meaning
<src_path>	string	src	where complab3d_cobrapy.py is found at run time. Only used by the COBRApy path, but it is imported by name, so the file must be there.
<input_path>	string	input	geometry and metabolic model files
<output_path>	string	output	VTK and checkpoint files. Must exist; the code does not create it.

23.2 <simulation_mode>

Tag	Type	Default	Meaning
<biotic_mode>	bool	true	false skips the whole <microbiology> block and runs pure chemistry
<enable_kinetics>	bool	true	run defineKinetics.hh
<enable_abiotic_kinetics>	bool	false	run defineAbioticKinetics.hh
<enable_fba_glpk>	bool	false	global switch for GLPK. Needs -DENABLE_GLPK=ON.
<enable_fba_cobrapy>	bool	false	global switch for COBRApy. Needs -DENABLE_COBRAPY=ON.
<enable_surrogate>	bool	false	global switch for the surrogate network
<fba_concentration_basis>	string	free	free or total. With speciation on, whether the uptake bound is built from the free ion or the total.
<glpk_lpsolver>	int	1	1 simplex, 2 interior point. GLPK only.

Tag	Type	Default	Meaning
<glpk_save_problem>	bool	false	dump the LP of the first solve, for debugging
<enable_validation_diagnostics>	bool	false	extra consistency printing at start-up. Worth switching on while setting a case up.

The two gates must agree. A microbe asking for glpk while <enable_fba_glpk> is false terminates the run with a message. So does a switch that is on in an executable built without the dependency. That is deliberate: you cannot silently get different biology from the one you asked for.

23.3 <LB_numerics>

23.3.1 <domain>

Tag	Type	Default	Meaning
<nx> <ny> <nz>	int	—	domain size in voxels. Mandatory.
<dx>	float	—	voxel size, in <unit>
<unit>	string	um	m, mm or um
<characteristic_length>	float	ny	length scale for the Péclet and Reynolds numbers, in voxels
<filename>	string	—	geometry file, inside <input_path>

23.3.2 <material_numbers>

Tag	Type	Default	Meaning
<pore>	int list	2	open fluid. A list, so several codes can be pore.
<solid>	int	0	no dynamics at all
<bounce_back>	int	1	no-slip wall
<microbe>n	int list	—	voxels where microbe n is seeded. Its presence also marks the organism as a biofilm former.

23.3.3 Flow and time stepping

Tag	Type	Default	Meaning
<delta_P>	float	0	pressure drop driving the flow, lattice units
<Peclet>	float	0	0 means no flow: pure diffusion, and the Navier–Stokes solver never runs

Tag	Type	Default	Meaning
`<tau>`	float	0.8	lattice-Boltzmann relaxation time. Values below about 0.55 are unstable.
`<track_performance>`	bool	`false`	timing output. Suppresses all VTI and checkpoint writing.
`<ns_max_iT1>`	int	100000	flow solver step cap, first solve
`<ns_max_iT2>`	int	100000	the same for later re-solves, after the geometry changes
`<ns_converge_iT1>`	float	1e-8	"flow is steady" tolerance, first solve
`<ns_converge_iT2>`	float	1e-6	the same, later
`<ns_update_interval>`	int	1	re-solve the flow every N reaction steps
`<ade_update_interval>`	int	1	apply reactions every N transport steps
`<ade_max_iT>`	int	1e7	reaction and transport steps to run
`<ade_converge_iT>`	float	1e-8	stop early if the chemistry stops changing

23.4 `<chemistry>`

Tag	Type	Default	Meaning
`<number_of_substrates>`	int	—	must equal the number of `<substrateN>` blocks
`<fix_concentration>`	bool list	all false	one per substrate. Marks an inexhaustible reservoir: read but never drawn down. Accepts `yes/no` and `1/0`.
`<fix_lower_bounds>`	bool list	all false	one per substrate. Clamps the FBA uptake bound to what is locally present.

23.4.1 Each `<substrateN>`

Tag	Type	Default	Meaning
`<name_of_substrates>`	string	—	used in output and to match `<components>`
`<initial_concentration>`	float	0	mol/L everywhere at $t = 0$
`<substrate_diffusion_>` `<coefficients>/<in_pore>`	float	1e-9	m^2/s in open water
…/`<in_biofilm>`	float	2e-10	m^2/s inside biofilm
`<immobile>`	bool	`false`	`true` means never advects or diffuses; receives only its reaction source. How minerals are represented.
`<left_boundary_type>`	string	—	`Dirichlet` or `Neumann`

Tag	Type	Default	Meaning
`<left_boundary_condition>`	float	0	the value: concentration for Dirichlet, flux for Neumann
`<right_boundary_type>`	string	—	as above
`<right_boundary_condition>`	float	0	as above

23.5 `<microbiology>`

Tag	Type	Default	Meaning
`<number_of_microbes>`	int	0	must equal the number of `<microbeN>` blocks
`<maximum_biomass_density>`	float	—	gDW/L at which a voxel is full. Mandatory with CA.
`<thrd_biofilm_fraction>`	float	—	fraction above which a voxel counts as biofilm. Mandatory with CA.
`<CA_method>`	string	`fraction`	how the cellular automaton shares excess biomass

23.5.1 Each `<microbeN>`: always

Tag	Type	Default	Meaning
`<name_of_microbes>`	string	—	used in output
`<solver_type>`	string	—	`CA`, `FD` or `LBM`. Mandatory.
`<reaction_type>`	string	`kinetics`	see the table below
`<initial_densities>`	float list	—	gDW/L. One entry per material number the organism is seeded in.
`<decay_coefficient>`	float	0	1/s, first order
`<half_saturation_constants>`	float list	—	mol/L, one per substrate. Feeds the FBA and surrogate uptake bounds.
`<maximum_uptake_flux>`	float list	all 0	the **kinetics** path's Vmax, mol substrate per mol cells per second
`<left_boundary_type>` …	—	Neumann 0	as for substrates

23.5.2 Each `<microbeN>`: conditionally

Tag	Required when	Meaning
`<viscosity_ratio_in_biofilm>`	the organism has a `<microbeN>` material entry	ratio used for the flow through biofilm. Must be present for *exactly* the seeded organisms; the parser counts them.
`<biomass_diffusion_> <coefficients>`	`solver_type` is FD	m^2/s, `<in_pore>` and `<in_biofilm>`
`<model_filename>`	`reaction_type` uses FBA, unless `<model_source>` supplies it	the metabolic model, inside `<input_path>`. May be an SBML file (`iJO1366.xml`) or the `extractMM.py` matrix format; the root element decides which. A bare stem gets `.xml` appended, as it always did.
`<exchange_reaction_indices>`	FBA, unless `<exchange_reaction_names>` is given	one per substrate; `-1` (or `-99`) for one the organism does not exchange. Correct only for the exact model revision it was written against.
`<exchange_reaction_names>`	alternative to the above; needs an SBML model	one reaction name per substrate, `none` for one the organism does not exchange. Resolved against the model at start-up, so a wrong name stops the run and suggests near matches. Wins if both are given.
`<fba_maximum_uptake_flux>`	FBA or surrogate	the **metabolic** path's Vmax, mmol/gDW/h. Separate from `<maximum_uptake_flux>`.
`<substrate_lower_bounds>`	FBA	hard floor on each exchange flux; eating is negative
`<substrate_upper_bounds>`	FBA	hard ceiling
`<objective_direction>`	FBA	`maximize` or `minimize`, default maximize
`<biomass_molar_mass>`	FBA or surrogate	gDW per mol of cells, default 24.6
`<equate_bounds>`	optional, FBA	`yes` forces uptake to exactly the bound instead of letting the solver choose anything up to it. Default `no`.
`<constraint_indices>`	optional, FBA	extra reactions to re-bound at load time
`<constraint_lower_bounds>`	with the above	same length
`<constraint_upper_bounds>`	with the above	same length

23.5.3 <reaction_type> spellings

Accepted	Meaning
`none no 0`	no metabolism
`kinetics kns 1`	`defineKinetics.hh`
`glpk fba fba_glpk 2`	flux balance analysis through GLPK
`surrogate srg ann 3`	the trained network
`cobrapy cpy fba_cobrapy 4`	flux balance analysis through COBRApy
`glpk_and_kinetics 5`	both, rates added
`surrogate_and_kinetics 6`	both
`cobrapy_and_kinetics 7`	both

23.6 <model_source> and friends

Top level, inside <parameters>. Chapter 15 explains the search order.

Tag	Type	Default	Meaning
<model_source>	string	—	`bigg:<id>`, e.g. `bigg:iJO1366`. Absent means every FBA microbe supplies its own <model_filename>.
<model_bundle>	string	`models`	where the gzipped bundled models live, relative to the working directory
<model_cache>	string	`input`	where the unpacked `.xml` is put, and looked for first
<allow_download>	bool	`false`	permit fetching over the network when the model is neither cached nor bundled. Compute nodes usually cannot, which is why this is off.

Who gets the file. Every microbe whose <reaction_type> is metabolic — FBA *or* surrogate — and that did not name its own <model_filename>. A surrogate microbe needs it too, because that is what <train_if_missing> sweeps.

23.7 Geometry generation, inside <domain>

All optional. Absent means <filename> is read as before. Chapter 16 has the details.

Tag	Type	Default	Meaning
<generate>	string	—	`channel`, `spheres`, `cylinders`, `random`, `fracture` or `layered`
<porosity>	real	0.5	target open fraction, for `spheres`, `cylinders` and `random`

Tag	Type	Default	Meaning
`<grain_radius>`	real	3	voxels, for `spheres` and `cylinders`
`<grain_count>`	int	—	fix the number of grains instead of the porosity
`<aperture>`	real	—	voxels, for `fracture`
`<roughness>`	real	0	fracture wall roughness, 0 to 1
`<layers>`	int	—	for `layered`
`<seed>`	int	1	the generator is deterministic given this
`<walls>`	string	—	axes to wall off, e.g. `y` or `yz`
`<write_geometry>`	string	`generated_geometry.dat`	where to record what was generated
`<import_raw>`	string	—	a flat binary volume to threshold instead. Mutually exclusive with `<generate>`.
`<raw_dtype>`	string	`uint8`	`uint8`, `uint16`, `int32`, `float32`, `float64`
`<threshold>`	real	128	at or above this is one phase, below is the other
`<invert>`	bool	`false`	swap which side of the threshold is pore

The run stops on a sealed domain. Not percolating along x is fatal, by design. See chapter 16.

23.8 `<surrogate>`

Top level, inside `<parameters>`. Chapter 17 explains what happens at start-up.

Tag	Type	Default	Meaning
`<enabled>`	bool	`false`	the whole block is ignored when this is off
`<weights_file>`	string	—	a `.srg` file. Required when the block is enabled.
`<train_if_missing>`	bool	`false`	fit one now if the file is not there, instead of stopping
`<microbe>`	int	−1	which microbe the network is for. −1 means all of them.

Inside `<train>`, all optional except the inputs:

Tag	Type	Default	Meaning
`<input0>` … `<input7>`	string	—	`<substrate> <low> <high> [log]`. At least one when training. Three at most — past that the samples needed grow faster than the fit improves.
`<samples>`	int	2000	points in the sweep
`<layers>`	ints	`10 10`	hidden layer sizes

Tag	Type	Default	Meaning
`<restarts>`	int	5	independent fits; the best is kept
`<epochs>`	int	4000	Adam iterations per restart
`<verify_points>`	int	512	fresh points, re-solved by the LP, that the network never saw
`<seed>`	int	1	the fit is reproducible given this, on every compiler
`<log_output>`	bool	`false`	fit $\log_{10}$ of the growth rate

The training range is not advice. It is written into the `.srg` file and enforced at run time: evaluations outside it are clamped to the edge and counted, and a run that clamps more than 5% of the time says so at the end. Set the ranges from what the simulation will actually visit — the upper end is `<fba_maximum_uptake_flux>`, because no voxel can ever request more.

23.9 `<diagnostics>`

Top level, inside `<parameters>`. Chapter 18 has the details.

Tag	Type	Default	Meaning
`<enabled>`	bool	`false`	the whole block is ignored when this is off
`<summary_csv>`	string	`summary.csv`	written inside `<output_path>` unless the path is absolute
`<interval>`	int	0	steps between rows. 0 follows the VTI interval.
`<tolerance>`	real	`1e-6`	relative drift above which a conserved sum is reported as failed
`<conserve>` `<conserve1>` `...<conserve31>`	string	—	a sum of substrate names, `+` separated, that ought to be constant

23.10 `<equilibrium>`

Tag	Type	Meaning
`<enabled>`	bool	default `false`
`<components>`	string list	master species. Names must match `<name_of_substrates>`.
`<stoichiometry>`/`<species>`*n*	float list	one row per substrate, one number per component. A row of zeros excludes the species.
`<logK>`/`<species>`*n*	float	formation constant. Components take 0.

23.11 <precipitation>

Tag	Type	Default	Meaning
<enabled>	bool	false	
<solid_substrate>	int	—	index of the mineral substrate. Must be <immobile>.
<max_precipRho>	float	—	mol/L of a full voxel; $1000/V_m$ with V_m in cm^3/mol
<surface_only>	int	1	1 grows only on wall-adjacent voxels (heterogeneous nucleation); 0 grows anywhere
<perm_ratio>	float	0	0 makes a filled voxel an impermeable wall
<update_interval>	int	—	steps between geometry re-checks. Each check that changes anything re-solves the flow.

23.12 <dissolution>

Tag	Type	Default	Meaning
<enabled>	bool	false	independent of <precipitation>
<reopen_fraction>	float	0.9	a voxel reopens below this fraction of full. Hysteresis; see below.
<surface_only>	int	1	1 dissolves only wetted voxels
<update_interval>	int	—	steps between geometry re-checks
<phase>n/<name>	string	phase$n$	label, for output
<phase>n/<material_number>	int	—	which mask code this phase occupies
<phase>n/<substrate>	int	—	which <immobile> substrate holds its inventory
<phase>n/<full_density>	float	—	mol/L when completely full
<phase>n/<initial_fill>	float	0	mol/L at $t = 0$. Non-zero for an original grain; 0 for a precipitate.
<phase>n/<is_precipitate>	bool	false	marks the phase that <precipitation> creates, so its voxels can later reopen

Only declared phases dissolve. Anything without a <phaseN> block can never dissolve, and that is deliberate. If "a solid voxel with no mineral left becomes pore" were the rule, an inert quartz grain — which never held any mineral — is already below any threshold, and your grain pack would dissolve on the first step.

23.13 <IO>

Tag	Type	Default	Meaning
<read_NS_file>	bool	false	restart the flow from a checkpoint
<read_ADE_file>	bool	false	restart the chemistry from a checkpoint
<ns_filename>	string	nsLattice	
<mask_filename>	string	maskLattice	
<subs_filename>	string	subsLattice	index and iteration appended
<bio_filename>	string	bioLattice	index and iteration appended
<save_VTK_interval>	int	1000	steps between snapshots
<save_CHK_interval>	int	1e6	steps between restart points
<debug_updRxn>	int	0	1 prints reaction-application debug lines

CHAPTER 24

When something goes wrong

24.1 The run refuses to start

CompLB3D validates its input before doing any work and stops with a named message. That is by design: a run that starts and produces nonsense is worse than one that refuses.

Message contains	Cause and fix
`The length of ...does not match`	A vector is the wrong length. The message names the tag. Almost always a substrate or microbe added without updating every per-substrate list.
`reaction_type ...is not defined`	A spelling the parser does not accept. See the table in Chapter 23.
`asks for reaction_type glpk but <enable_fba_glpk> is false`	The per-microbe choice and the global switch disagree. Set the switch, or change the microbe.
`is true but this executable was built without GLPK`	Rebuild with `-DENABLE_GLPK=ON`.
`solver_type ...is not defined`	Not `FD`, `CA` or `LBM`.
`must be defined when solver_type is Finite Difference`	FD needs `<biomass_diffusion_coefficients>`.
`biofilm_permeability_ratio vector does not match`	`<viscosity_ratio_in_biofilm>` is present for a different set of organisms than the ones seeded under `<material_numbers>`. It must be exactly the seeded ones.
`<S> has ...entries but nmet*nrxn =`	The metabolic model file is truncated or misgenerated. Re-run `extractMM.py`.
`<exchange_reaction_indices> entry ...is outside`	An index past the end of that organism's model.
`must be defined when solver_type is Cellular Automata`	CA needs `<thrd_biofilm_fraction>`.

Catch these before you build. `examples/validateExamples.py` applies most of these rules to an input file in about a second, without compiling anything. Point it at a folder containing your `CompLaB.xml`.

24.2 The run starts but the answer is wrong

Symptom	Usual cause
`[NEG!]` warnings	A reaction removed more than was present in one step. Reduce `<ade_update_interval>`, or your rate constant is too large — check the per-second conversion.
nothing grows	Several possibilities, in order of likelihood: rate constants 86400 times too small (a per-day constant used directly); `<enable_kinetics>` false; the organism seeded on a material number that does not occur in the geometry; substrate never reaching it.
biomass explodes	No decay, no maximum density, or growth not limited by anything. Check `<maximum_biomass_density>`.
the answer depends on z	In a quasi-2D setup it must not. Usually the geometry file was written in the wrong loop order.
the geometry looks scrambled	The geometry file loop order. x fastest, then y, then z.
flow never converges	`<tau>` too close to 0.5; or `<delta_P>` so large the flow is not creeping; or the domain has no percolating path.
FBA growth is always zero	The uptake bounds are zero, or `<exchange_reaction_indices>` points at the wrong reactions, or the model's objective is not the biomass reaction.
surrogate returns nonsense	The run has left the network's training box. Run `inspectSurrogate.py` and compare with your `<fba_maximum_uptake_flux>`.
combined type double counts growth	The kinetics file is writing `bioR` as well as the FBA path. It must not.
porosity oscillates	`<reopen_fraction>` too close to 1. Try 0.9.

24.3 Performance

Symptom	What to do
speciation dominates the run	It will. It is the most expensive part of the code by a wide margin. Turn it off unless you need it.
COBRApy is very slow	Expected: one to two orders of magnitude slower than GLPK per voxel. Use GLPK for production and COBRApy as a cross-check.
FBA dominates	Fit a surrogate. Chapter 20.
more MPI ranks made it slower	Too few voxels per rank. See Chapter 21.
the flow re-solves constantly	`<update_interval>` in `<precipitation>` or `<dissolution>` is too small, or the porosity is chattering.

CHAPTER 25

Known limits

Stated plainly, because knowing them saves more time than working around them.

- **Speciation is ideal.** No activity corrections, no temperature dependence, no gas phase, no redox couples, no mineral saturation indices. Your log K values are conditional constants at your own ionic strength and 25 °C.
- **Dissolution is not driven by saturation state.** Because there are no solubility products, precipitation and dissolution are separate rate laws that happen to move the same mineral, rather than one reversible reaction.
- **Rate laws are compiled in.** Two problems with different chemistry need two binaries.
- **One fluid phase.** No unsaturated flow.
- **The `.vti` output has unit spacing**, so ParaView shows lattice coordinates rather than micrometres.
- **The shipped surrogate is one organism on two substrates** over one range. It is a demonstration of the mechanism, not a general-purpose solver option the way GLPK is.
- **Palabos v2.3.0 specifically.** Later versions change data-processor interfaces.
- **`src/complab3d_glpkcpp.hh` derives from GLPKMEX** / the Octave-Forge `glpkcc.cpp`, which is GPL-licensed. Its provenance is recorded in the file header; anyone redistributing should satisfy themselves this is correct for their situation.

CHAPTER 26

Where everything lives

Path	What it is
You will edit these	
`CompLaB.xml`	the run
`defineKinetics.hh`	your biotic rate laws, compiled in
`defineAbioticKinetics.hh`	your abiotic rate laws and the dissolution hook
`surrogateModel.hh`	the trained surrogate network
`CMakeLists.txt`	the Palabos path, lines 85 to 93
Reference and tools	
`CompLaB_reference_template.xml`	every tag, documented inline
`CompLaB_example_uranium_zhao.xml`	a full 95-species production case
`extractMM.py`	genome-scale model → matrix file
`comp.sh`	Slurm template
The test suite	
`examples/`	fifteen cases, one per capability
`examples/makeExamples.py`	regenerates them
`examples/validateExamples.py`	structural check, one second
`examples/runAllExamples.sh`	builds and runs all fifteen
`examples/runExamples.slurm`	the same, under Slurm
Surrogate training	
`surrogate_training/generateTrainingData.py`	diffusion-EBA sweep
`surrogate_training/trainSurrogate.py`	fit and export, no licence needed
`surrogate_training/trainSurrogate.m`	the MATLAB path
`surrogate_training/verifyExport.py`	compiles the export and checks it
`surrogate_training/inspectSurrogate.py`	prints a network's valid range
Source you are unlikely to touch	
`src/complab.cpp`	the driver: parse, set up lattices, time loop
`src/complab_functions.hh`	XML parsing and lattice setup
`src/complab3d_processors*.hh`	the data processors, one file per concern
`src/complab3d_metabolic.hh`	the optional metabolic layer
`src/complab3d_glpkcpp.hh`	the GLPK wrapper
`src/complab3d_pythonAPI.hh`	the COBRApy bridge, C++ side
`src/complab3d_cobrapy.py`	the COBRApy bridge, Python side

Path	What it is
`src/precipitationVOP.hh`	pore clogging
`src/dissolutionVOP.hh`	pore reopening

Reporting a problem. Open an issue at `https://github.com/shahram444/CompLB3D-lattice-Boltzmann-FBA-surrogate-COBRApy-GLPK` with your `CompLaB.xml`, the domain size, the cmake line and the terminal output. The start-up banner is verbose and usually contains the answer. If it is a crash, the smallest input that still reproduces it is worth more than a large one.